PANDUAN TEKNIS - BAHASA INDONESIA

CV Optimizer - Panduan Teknis

Panduan belajar untuk memahami struktur, alur request, penyimpanan data,

Prepared from the project documentation. See the Markdown source in docs/.

antrean AI, pengujian, dan operasi proyek CV Optimizer.

Edisi dokumentasi: Oktober 2026

> *Panduan ini menjelaskan implementasi yang ada di repository, bukan janji bahwa deployment produksi sudah selesai atau lolos uji kapasitas.*

1. Gambaran arsitektur

```
Browser
|
| HTTPS / JSON / multipart upload
v
React + Vite frontend
|
| REST API, cookie auth, polling job
v
Express API ----------------------> Google Gemini
|                                    ^
|                                    |
+--> PostgreSQL (Prisma)             |
|                                    |
+--> Redis (BullMQ queue) --> AI worker+
```

Frontend dan backend adalah aplikasi terpisah. API menerima request dan memvalidasi input; pekerjaan yang memerlukan AI dikirim ke Redis. Worker mengambil task, memanggil service/AI provider, lalu menyimpan hasil sementara di queue. Frontend membaca status melalui API.

Komponen runtime:

- Frontend: React, TypeScript, Vite, React Router, Tailwind CSS.
- API: Node.js, Express, TypeScript, Zod, Multer.
- Data relasional: PostgreSQL melalui Prisma ORM.
- Antrean dan rate limit lintas instance: Redis dan BullMQ.
- AI: Google Gemini di balik interface AIProvider.
- File: pdf-parse untuk PDF, mammoth untuk DOCX; PDF CV/cover letter dan DOCX CV dibuat server-side.
- Local orchestration: Docker Compose menjalankan Postgres, Redis, API, worker, dan frontend.

2. Peta repository

```
.
|-- README.md
|-- docker-compose.yml
|-- .env.example
|-- docs/
|   |-- TECHNICAL_GUIDE.md
|   |-- CV-Optimizer-Project-Overview.pdf
|   `-- Technical-Guide-ID.pdf
|-- frontend/
|   |-- src/App.tsx
|   |-- src/pages/
|   |-- src/components/
|   |-- src/context/
|   `-- src/lib/api.ts
`-- backend/
|-- src/app.ts
|-- src/server.ts
|-- src/routes/
|-- src/controllers/
|-- src/middleware/
|-- src/schemas/
|-- src/services/
|   |-- ai/
|   |-- cv/
|   |-- document/
|   |-- jobs/
|   `-- profile/
|-- src/utils/
|-- prisma/schema.prisma
|-- prisma/migrations/
`-- test/
```

Tanggung jawab layer backend

1. Routes memasang URL, middleware autentikasi, validasi/rate limit, dan controller.
2. Middleware menangani autentikasi, upload, rate limiting, not-found, dan pemetaan error.
3. Controllers mengubah request HTTP menjadi pemanggilan service lalu memilih response HTTP.
4. Schemas memvalidasi bentuk input dengan Zod.
5. Services memuat logika bisnis, integrasi model AI, pemrosesan profil, rekomendasi, dan pembentukan dokumen.
6. Prisma menyediakan akses terstruktur ke PostgreSQL.

Jaga batas layer ini: route sebaiknya tetap tipis, dan service tidak semestinya bergantung pada object request/response Express.

3. Alur utama: optimasi CV

Alur dari upload sampai unduh dokumen secara garis besar:

1. Pengguna masuk menggunakan Google Identity Services.

2. Frontend mengirim CV PDF/DOCX dan deskripsi pekerjaan ke POST /api/cv/analyze.
3. API memeriksa sesi, limit request, tipe/ukuran file, lalu membaca teks CV.
4. API memvalidasi task dan membuat reservasi kuota bulanan di PostgreSQL.
5. Payload task dienkripsi, lalu job ditambahkan ke BullMQ di Redis.
6. API mengirim 202 Accepted beserta jobId; frontend melakukan polling GET /api/ai/jobs/:jobId.
7. Worker mendekripsi input, memanggil service AI, mengenkripsi hasil untuk disimpan pada queue, lalu menyelesaikan reservasi kuota.
8. Frontend menerima CV terstruktur dan analisis, menampilkan draft yang dapat diedit, dan mengirim versi akhir untuk unduhan PDF/DOCX.

Analisis kecocokan adalah indikasi berbasis input, bukan prediksi peluang diterima. Pengguna harus mengoreksi informasi yang dihasilkan sebelum mengirimkan CV.

Aturan penggunaan AI

AIProvider adalah kontrak untuk ekstraksi CV, analisis, pembuatan CV, cover letter, dan persiapan wawancara. GeminiProvider menyediakan implementasi saat ini. Dengan demikian, controller dan route tidak harus mengetahui detail SDK Gemini.

Prompt memisahkan fakta dari CV, persyaratan lowongan, dan instruksi. Ada validasi/post-processing untuk membatasi informasi yang dibuat model, tetapi ini bukan jaminan bahwa semua keluaran benar. Review manusia tetap diperlukan.

Antrean, kuota, dan enkripsi job

- Nama antrean BullMQ saat ini: cv-optimizer-ai.
- API dan worker harus memakai Redis serta AI_JOB_ENCRYPTION_KEY yang sama.
- Payload job menggunakan AES-256-GCM dengan IV acak dan authentication tag.
- Key harus berupa base64 yang valid dari tepat 32 byte. Rotasi key perlu direncanakan; worker harus dapat membaca job lama yang masih tersimpan.
- Kuota default adalah 10 task per user per bulan UTC, dan dapat diubah melalui AI_MONTHLY_QUOTA.
- Reservasi kuota dibuat secara transaksional sebelum enqueue. Jika enqueue gagal atau semua percobaan worker gagal, slot dirilis; jika task berhasil, slot dikonsumsi.
- BullMQ dikonfigurasi dengan dua percobaan dan exponential backoff. Worker memiliki concurrency yang dapat diatur (AI_WORKER_CONCURRENCY, 1 sampai 32).
- Job yang berhasil disimpan sementara selama sekitar 10 menit; job gagal disimpan sampai 24 jam sesuai opsi queue saat ini. Ini bukan arsip hasil.

Redis harus memiliki availability dan persistence yang sesuai dengan kebutuhan antrean. Jangan anggap encrypted payload sebagai pengganti kontrol akses, backup, secret management, atau kebijakan retensi.

4. Autentikasi, data, dan error

Autentikasi

Frontend mengirim credential/cookie sesuai konfigurasi API; backend memverifikasi login Google dan menyediakan route sesi /api/auth/me. Route fitur AI utama mensyaratkan autentikasi. Beberapa route lowongan menerima akses opsional; periksa requireAuth / optionalAuth di route sebelum mengubah akses suatu endpoint.

FRONTEND_URL harus cocok persis dengan origin frontend yang diizinkan CORS. Produksi di belakang proxy juga harus mengatur TRUST_PROXY_HOPS sesuai jumlah hop tepercaya.

Model data

Schema Prisma saat ini meliputi:

- User: identitas akun dan relasi.
- CvVersion: CV terstruktur opsional yang terkait user.
- Application: lowongan, skor/analisis, status tracker, catatan, tanggal.
- UsageCredit: pemakaian dan limit kuota bulanan user.
- AiJobReservation: kaitan job queue dengan reservasi kuota.
- ApplicationStatus: applied, hr_interview, user_interview, technical_test, offer, rejected, ghosted, withdrawn.

Gunakan file migration Prisma yang ter-commit sebagai sumber perubahan schema. Untuk rilis, migration perlu dijalankan sekali sebagai langkah release, bukan serentak dari setiap replika API.

Bentuk error

AppError mendefinisikan status HTTP, pesan aman untuk pengguna, dan kode stabil, misalnya INVALID_FILE_TYPE, AI_REQUEST_TIMEOUT, QUOTA_EXCEEDED, dan VALIDATION_ERROR. Error handler pusat memformat response; jangan mengirim stack trace atau secret ke klien.

Saat menambah error baru:

1. Tambahkan kode ke union AppErrorCode bila perlu.
2. Buat factory di Errors atau lempar AppError dengan status yang sesuai.
3. Tambah test untuk status, code, dan bentuk response.
4. Pastikan log operasional cukup untuk diagnosa tetapi tidak memuat isi CV, token, atau data pribadi yang tidak perlu.

5. Peta API

Prefix semua endpoint adalah /api. Kolom akses di bawah menjelaskan middleware route yang sekarang, bukan jaminan authorization untuk deployment tertentu.

Sesi dan kesehatan

- GET /auth/me - baca sesi saat ini.
- POST /auth/google - tukar credential Google untuk sesi aplikasi.
- POST /auth/logout - logout.
- GET /health - pemeriksaan liveness sederhana.
- GET /ready - memeriksa koneksi database dan Redis.

CV dan tugas AI

- POST /cv/analyze - auth; upload cv dan jobDescription, antrekan ekstraksi dan analisis.

- POST /cv/generate - auth; antrekan CV yang disesuaikan.
- POST /cv/cover-letter - auth; antrekan cover letter.
- POST /cv/interview-prep, /cv/interview/start, /cv/interview/evaluate - auth; task latihan wawancara.

- POST /profile/import - auth; impor dari sumber yang didukung.
- POST /jobs/talent/bulk-screen - auth; screening CV yang diberikan pengguna dengan AI.

- POST /ai/jobs - auth; submit task tervalidasi ke antrean.
- GET /ai/jobs/:jobId - auth; status/hasil job hanya untuk pemilik job.
- POST /cv/pdf, /cv/docx, /cv/cover-letter/pdf - auth; unduh dokumen.

Submit AI mengembalikan 202 dan jobId; status job dapat berupa waiting, active, delayed, completed, atau failed. Frontend harus menampilkan loading, error, retry yang aman, dan hasil sukses secara berbeda.

Latihan wawancara menerima pilihan bahasa Indonesia atau Inggris. Dalam mode suara, SpeechSynthesis pada browser membacakan pertanyaan dan respons AI; SpeechRecognition browser mengubah jawaban lisan menjadi transkrip. Hanya transkrip yang dikirim aplikasi ke Gemini untuk evaluasi, tetapi pemrosesan audio oleh browser/perangkat mengikuti kebijakan penyedia masing-masing. Respons AI dapat menghasilkan pertanyaan lanjutan berdasarkan jawaban. Ini latihan suara bergiliran, bukan streaming percakapan real-time atau sistem proctoring; jangan menganggapnya dapat mencegah pengguna membuka contekan.

Tracker, profil, dan lowongan

- /applications - CRUD tracker dan metrik aplikasi.
- DELETE /profile/account - hapus akun saat ini dan data terkait sesuai implementasi backend.

- /jobs/recommendations, /jobs/analyze-custom, /jobs/outreach-pitch, /jobs/talent/search, dan /jobs/talent/invite-template - utilities lowongan/recruiter; periksa route untuk status autentikasinya.

Saat ini pencarian talent tidak boleh dipahami sebagai pencarian katalog kandidat nyata. Tidak ada database kandidat berizin/consent yang digunakan oleh fitur tersebut.

Rekomendasi lowongan mengurutkan hingga 100 listing dengan kecocokan skill CV, kemudian memprioritaskan listing yang secara eksplisit menyebut visa sponsorship. Sumber lokal Indonesia mencakup Kalibrr; pencarian negara Malaysia, Singapura, dan Australia menggunakan JOOBLE_API_KEY opsional; sumber remote global memiliki pembatasan negara masing-masing. Status visa adalah deteksi

teks sederhana: "belum disebut" tidak berarti sponsorship tersedia, dan kandidat tetap harus memverifikasi syarat dengan perusahaan. Pengguna dapat memilih beberapa listing untuk menyiapkan CV tailored dan cover letter secara berurutan. Satu paket memakai tiga AI task per listing (analisis CV, CV tailored, dan cover letter), sehingga jumlahnya dibatasi oleh sisa kuota akun. Hasil batch perlu ditinjau dan diunduh sebelum sesi halaman ditinggalkan; hasil belum disimpan sebagai lamaran di tracker secara otomatis. Materi lamaran harus ditinjau dan dikirim lewat situs resmi; belum ada submit otomatis ke ATS eksternal.

Extension Chrome/Edge opsional berada di browser-extension/. Extension Manifest V3 hanya berjalan pada halaman HTTPS Greenhouse dan Lever yang diizinkan, membaca paket JSON lokal hasil batch, lalu mengisi field kontak dan cover letter yang dikenali. Data paket tidak dikirim ke backend atau disimpan oleh extension. Pengguna tetap mengunggah CV, menjawab pertanyaan form, memeriksa seluruh isi, dan menekan tombol submit sendiri. Setelah benar-benar submit, pengguna menandai hasil batch untuk menyimpan lowongan, status "terkirim", dan skor kecocokan ke tracker/dashboard. Skor tersebut bukan probabilitas diterima. Panduan instalasi lokal tersedia di browser-extension/README.md.

6. Menjalankan proyek

Docker Compose (cara tercepat)

Di PowerShell dari direktori root:

```
Copy-Item .env.example .env
```

Isi minimal GEMINI_API_KEY, JWT_SECRET, dan AI_JOB_ENCRYPTION_KEY, lalu:

```
docker compose up --build
```

Compose menyalakan Postgres, Redis, API, worker, dan frontend; API menerapkan migration yang ter-commit saat start. Port lokal dipublikasikan pada loopback. Konfigurasi compose saat ini menetapkan NODE_ENV=development; ini stack pengembangan lokal, bukan blueprint produksi.

Untuk berhenti tanpa menghapus data:

```
docker compose down
```

docker compose down -v menghapus volume database dan Redis. Gunakan hanya ketika data lokal boleh dihapus.

Pengembangan tanpa container

1. Siapkan PostgreSQL dan Redis.
2. Jalankan npm ci dari backend dan frontend.
3. Salin .env.example ke backend/.env, dan frontend/.env.example ke frontend/.env; isi URL dan credential yang sesuai.
4. Dari backend, jalankan npm run prisma:generate, lalu

npm run prisma:migrate untuk database development baru.

5. Buka terminal terpisah:

```
# backend API
cd backend
npm run dev

# backend worker - terminal lain
cd backend
npm run worker:dev

# frontend - terminal lain
cd frontend
npm run dev
```

Nilai frontend VITE_API_URL dan backend FRONTEND_URL harus berpasangan dengan origin lokal yang digunakan. OAuth Google perlu mengizinkan origin tersebut juga.

Environment variables

Variable | Dipakai oleh | Tujuan

DATABASE_URL | API, worker, Prisma | PostgreSQL connection string

REDIS_URL | API, worker | BullMQ dan rate limit bersama

JWT_SECRET | API | Menandatangani token sesi

AI_JOB_ENCRYPTION_KEY | API, worker | AES-256-GCM untuk payload job

GEMINI_API_KEY | API bila perlu, worker | Credential Google Gemini

GEMINI_MODEL | API/worker | Nama model yang dikonfigurasi

JOOBLE_API_KEY | API (opsional) | Pencarian listing per negara di Jooble

AI_MONTHLY_QUOTA | API/worker | Kuota task bulanan per user

AI_WORKER_CONCURRENCY | worker | Jumlah task paralel (1-32)

FRONTEND_URL | API | Origin CORS yang diizinkan

VITE_API_URL | frontend build | URL API yang disematkan saat build

GOOGLE_CLIENT_ID | API/compose | Google OAuth Web Client ID

VITE_GOOGLE_CLIENT_ID | frontend build | Google OAuth Client ID frontend

TRUST_PROXY_HOPS | API | Proxy tepercaya saat NODE_ENV=production

Lihat .env.example, backend/.env.example, dan frontend/.env.example sebagai referensi terkini. Jangan commit file .env. Frontend VITE_* bukan secret: nilainya dapat dibaca di bundle browser.

7. Migrasi database dengan aman

- Gunakan npm run prisma:migrate hanya untuk membuat/menerapkan migration di development.

- Jalankan npm run prisma:deploy sebagai langkah release terhadap database deployment.

- Backup database sebelum perubahan schema produksi dan siapkan prosedur restore.

- Jangan menggunakan prisma db push sebagai pengganti migration release.
- Database lama yang dibuat dengan db push perlu dibandingkan dengan migration awal dan dibackup sebelum migration history di-baseline.

8. Pengujian dan kualitas

Backend:

```
cd backend
npm test
```

Script backend membangun TypeScript lalu menjalankan test Node.js di backend/test/*.test.cjs. Coverage saat ini antara lain schema validasi, JWT, konfigurasi production, error API, enkripsi job, health check, rekomendasi lowongan, privasi log AI, dan PDF cover letter.

Frontend:

```
cd frontend
npm run check
```

check menjalankan Oxlint dan production build (TypeScript + Vite). Workflow GitHub Actions menjalankan pemeriksaan otomatis untuk push dan pull request.

Untuk perubahan fitur, tambah test yang memeriksa perilaku yang penting bagi pengguna, bukan hanya implementasi internal. Uji kondisi gagal AI, timeout, input invalid, pengguna yang tidak berwenang, dan tampilan loading/error.

9. Deployment dan kesiapan produksi

Production memerlukan setidaknya frontend, API, satu atau lebih worker, PostgreSQL terkelola, dan Redis terkelola. Pisahkan perintah migration sebagai langkah release satu kali. API dan worker harus memiliki kunci enkripsi serta konfigurasi provider yang konsisten.

Checklist sebelum rilis:

- Domain HTTPS, CORS dan Google OAuth client menggunakan origin produksi yang tepat.
- Secret unik disimpan di secret manager; jangan gunakan nilai contoh.
- Backup otomatis PostgreSQL dan prosedur restore sudah diuji.
- Redis memiliki persistence/availability sesuai persyaratan recovery queue dan tidak terekspos publik tanpa kontrol jaringan.
- Alert biaya/kuota Gemini, error API/worker, queue depth, dan kesehatan dependency sudah dikonfigurasi.
- Tes end-to-end mencakup login, upload, antrean, retry, download, logout,

dan hapus akun.

- Beban diuji terhadap tier hosting aktual sebelum mengklaim skala pengguna.
- Notice privasi, periode retensi, contact operator, lokasi hosting, dan ketentuan pemrosesan oleh penyedia AI sudah ditinjau untuk pasar tujuan.

Build dan unit test bukan load test. Jumlah pengguna yang dapat ditangani bergantung pada request mix, latensi dan batas provider, concurrency worker, ukuran database/Redis, serta konfigurasi hosting.

Biaya Gemini jika aplikasi dibuat gratis

Google menyediakan free tier untuk model dan batas pemakaian tertentu; model, rate limit, dan harga dapat berubah. Di aplikasi publik gratis, pengguna tidak otomatis membayar Google: biaya API ditanggung pemilik project/kunci Gemini jika pemakaian masuk ke tier berbayar dan billing aktif. Cek harga Gemini API dan rate limit project sebelum rilis.

AI_MONTHLY_QUOTA membatasi jumlah task per pengguna, bukan jumlah panggilan model atau total biaya. Satu task dapat memerlukan beberapa panggilan Gemini dan percobaan/fallback model. Karena itu, jangan menjanjikan layanan gratis tanpa batas hanya berdasarkan kuota aplikasi atau free tier; pantau pemakaian dan biaya pada project Google, pasang alert, dan tetapkan kuota aplikasi yang sesuai dengan anggaran operator.

10. Batasan produk yang perlu dipahami

- Hasil generatif dapat salah atau mengubah nuansa kalimat; verifikasi semua fakta CV dan cover letter.

- Match score bukan estimasi peluang lolos atau keputusan ATS.
- Pemrosesan AI mengirim konten yang dibutuhkan ke Google Gemini dan tunduk pada ketentuan provider. Baca halaman privasi aplikasi sebelum upload data.

- Upload disimpan sementara untuk diproses; deployment operator perlu membatasi akses direktori upload dan punya kebijakan pembersihan.

- Tracker dan akun menggunakan data persisten; behavior penyimpanan jangan disimpulkan hanya dari hasil analisis CV yang tidak disimpan.

- Sourcing kandidat belum terhubung ke database kandidat dengan consent.
- Antarmuka terutama Bahasa Indonesia; pengalaman bahasa Inggris belum sepenuhnya dilokalkan.

11. Jalur belajar yang disarankan

Ikuti satu alur vertikal supaya hubungan frontend, API, service, dan test terlihat jelas:

1. Baca frontend/src/App.tsx untuk route publik, proteksi autentikasi, dan lazy loading halaman.

2. Baca halaman fitur terkait dan frontend/src/lib/api.ts untuk melihat

pengiriman request, polling, serta feedback UI.

3. Ikuti endpoint di backend/src/routes/, lalu controller pada backend/src/controllers/.

4. Pelajari schema Zod, middleware auth/error, kemudian service domain yang dipanggil controller.

5. Untuk AI task, baca backend/src/services/ai/aiTasks.ts, aiJobProcessor.ts, aiWorker.ts, dan AIProvider.ts.

6. Cocokkan perilaku itu dengan migration Prisma dan test yang relevan.

Latihan perubahan bertahap:

- Tambah validasi input dengan schema Zod dan test untuk batas valid/invalid.
- Tambah satu kode error yang stabil melalui AppError dan uji response-nya.
- Tambah field pada tracker dengan migration Prisma serta UI/API/tests.
- Tambah indikator progress AI yang tidak membingungkan waiting, active, completed, dan failed.
- Implementasikan provider AI alternatif dengan interface yang sama, tanpa memasukkan API key ke frontend.

Untuk setiap perubahan, jalankan test backend yang berkaitan, npm test, npm run check frontend, lalu tinjau git diff --check dan diff sebelum commit.

12. Istilah singkat

- API: antarmuka HTTP yang dipakai frontend dan worker.
- BullMQ: library job queue Node.js berbasis Redis.
- CORS: aturan browser yang menentukan origin frontend mana yang dapat memanggil API.
- Migration: perubahan schema database yang tercatat dan dapat diterapkan berurutan.
- Readiness: indikasi bahwa service siap menerima pekerjaan dan dependency yang dicek tersedia.
- Worker: proses terpisah yang menjalankan task AI dari queue.